



PRÉ-RENTREE 2026-2027

SÉANCE 3 - FONCTIONS, PARAMÈTRES, RETOUR ET TESTS

Dossier personnalisé - Ahmad BELDI - durée : 2 heures

Parcours : Fondations guidées

Objectif personnel de la séance 3 : distinguer clairement **paramètre**, **argument**, **valeur renvoyée** et **None**, puis écrire **trois fonctions autonomes** accompagnées de tests exécutables.

Production à conserver : le fichier `S3_fonctions_tests_Ahmad.py`, contenant `est_pair`, `maximum_deux`, `compte_occurrences` et au moins **huit assertions**.

Ma méthode de travail aujourd'hui

1. Je lis la spécification et je nomme les **entrées** et la **sortie**.
2. J'écris au moins un résultat attendu **avant** de coder.
3. Je code une fonction courte, puis je l'exécute immédiatement.
4. Je conserve les assertions qui réussissent : elles deviennent une preuve.
5. Si un test échoue, je ne modifie qu'une seule chose avant de relancer.

1. Feuille de route des 120 minutes

Temps	Travail	Preuve attendue
0–10 min	Rituel sans ordinateur : appel de fonction et retour.	Deux prévisions écrites.
10–30 min	Paramètres, arguments et portée locale.	Anatomie d'une fonction complétée.
30–48 min	Différence entre <code>print</code> , <code>return</code> et <code>None</code> .	Valeurs affichées et renvoyées distinguées.
48–65 min	Contrat d'une fonction et fabrique de tests.	Huit résultats attendus préparés.
65–70 min	Pause.	–
70–100 min	Écriture de trois fonctions dans le fichier Python.	Trois fonctions exécutables.
100–112 min	Assertions et débogage ciblé.	Huit tests verts et un bug expliqué.
112–120 min	Revue du module et exit ticket.	Explication individuelle concise.

Critères de réussite : trois fonctions terminées ; aucun mélange entre afficher et renvoyer ; comportement de chaque fonction expliqué ; huit assertions exécutées ; une correction réalisée à partir d'un test qui échoue.

2. Rituel : prévoir avant d'exécuter - 10 min

Consigne unique : pour chaque programme, écrire d'abord la prévision. Exécuter seulement après avoir rempli les cases, puis noter la comparaison.

Rituel 1 - afficher n'est pas renvoyer

```
def double(x):
    print(2 * x)

r = double(5)
print("r =", r)
```

Question	Réponse avant exécution
Quelle valeur est affichée dans la fonction ?	
Quelle ligne est ensuite affichée par le dernier print ?	
Quelle valeur est stockée dans r ?	
Pourquoi ?	
Certitude : ☐ 1 ☐ 2 ☐ 3 ☐ 4	Contrôle : _____

Rituel 2 - paramètre et argument

```
def aire_rectangle(longueur, largeur):
    aire = longueur * largeur
    return aire

resultat = aire_rectangle(8, 3)
```

Élément demandé	Réponse
Nom de la fonction	
Paramètres	
Arguments lors de l'appel	
Variable locale	
Valeur renvoyée et stockée	

Rituel 3 - réactivation de la séance 2

Sans exécuter, compléter la table pour l'appel `compte_positifs([-2, 4, 0, 7])`.

```
def compte_positifs(valeurs):
    compteur = 0
    for valeur in valeurs:
        if valeur > 0:
            compteur += 1
    return compteur
```

Tour	valeur	compteur après	Condition vraie ?
1			
2			
3			
4			

3. Anatomie d'une fonction et portée locale - 20 min

Repère. Le **paramètre** est un nom écrit dans la définition. L'**argument** est la valeur fournie lors de l'appel. Une variable créée dans la fonction est **locale** : elle n'existe que pendant cet appel, sauf usage explicite et exceptionnel d'une variable globale.

A. Compléter la carte d'identité

On étudie :

```
def nombre_paires(valeurs):
    compteur = 0
    for valeur in valeurs:
        if valeur % 2 == 0:
            compteur += 1
    return compteur
```

Rubrique	Réponse précise
Nom de la fonction	
Paramètre	
Type d'entrée prévu	
Variables locales	
Type de sortie prévu	
Instruction qui transmet la sortie	

B. Portée locale : deux variables portant le même nom

```
compteur = 99

def nombre_paires(valeurs):
    compteur = 0
    for valeur in valeurs:
        if valeur % 2 == 0:
            compteur += 1
    return compteur

r = nombre_paires([2, 5, 8])
print(compteur, r)
```

1. Prévoir la ligne affichée : _____
2. Le `compteur` de la fonction et le `compteur` extérieur désignent-ils le même état ? Justifier.

3. Entourer dans le code la variable locale ; souligner la variable extérieure.

C. Interface avant code

Pour une fonction `est_pair(n)`, compléter sans écrire encore le corps.

Entrée	Sortie	Exemple d'appel

Aides graduées - une seule à la fois : A - nommer les entrées et la sortie ; B - distinguer `print` et `return` ; C - repérer les variables locales ; D - écrire la précondition et la postcondition ; E - ajouter un cas normal, un cas limite et un cas invalide.

4. print, return et None - 18 min

A. Comparer deux fonctions

Exécuter exactement le programme après avoir rempli la colonne « Prévvision ».

```
def afficher_double(n):  
    print(2 * n)  
  
def renvoyer_double(n):  
    return 2 * n  
  
a = afficher_double(6)  
b = renvoyer_double(6)
```

Question	Prévvision	Résultat observé
Que voit-on à l'écran pendant afficher_double(6) ?		
Valeur de a		
Valeur de b		
Peut-on calculer b + 1 ?		
Peut-on calculer a + 1 ?		

Point de vigilance. `print` produit un affichage destiné à l'humain. `return` transmet une valeur au programme appelant. Lorsqu'aucun `return` n'est exécuté, la valeur renvoyée est `None`.

B. Correction minimale

La fonction suivante affiche une moyenne mais ne permet pas de la réutiliser.

```
def moyenne_affichee(a, b):  
    moyenne = (a + b) / 2  
    print(moyenne)  
  
m = moyenne_affichee(12, 16)
```

1. Valeur actuelle de `m` : _____
2. Modifier **une seule ligne** pour que `m` reçoive la moyenne.

3. Écrire ensuite une ligne qui affiche `m` sans modifier la fonction.

C. Checkpoint

Compléter en une phrase : « Une fonction doit utiliser `return` lorsque... »

5. Contrat et tests : préparer avant de coder - 17 min

Contrat minimal. Pour chaque fonction, écrire : les entrées attendues, la sortie garantie et un comportement explicite pour les cas particuliers. Une assertion vérifie qu'un résultat prévu est bien obtenu.

A. Les trois spécifications de la séance

Fonction	Entrées	Sortie garantie
<code>est_pair(n)</code>	un entier <code>n</code>	un booléen indiquant si <code>n</code> est pair
<code>maximum_deux(a, b)</code>	deux nombres	le plus grand des deux, y compris s'ils sont égaux ou négatifs
<code>compte_occurrences(valeurs, cible)</code>	une liste et une valeur cible	le nombre d'éléments égaux à la cible ; 0 pour une liste vide

B. Prévoir huit résultats avant le code

N°	Appel	Résultat attendu	Test écrit ?
1	<code>est_pair(8)</code>		☐
2	<code>est_pair(7)</code>		☐
3	<code>est_pair(-4)</code>		☐
4	<code>maximum_deux(3, 9)</code>		☐
5	<code>maximum_deux(-4, -9)</code>		☐
6	<code>maximum_deux(5, 5)</code>		☐
7	<code>compte_occurrences([2, 5, 2, 2], 2)</code>		☐
8	<code>compte_occurrences([], 4)</code>		☐

C. Transformer une prévision en assertion

Compléter :

```
assert est_pair(8) == _____
assert maximum_deux(-4, -9) == _____
assert compte_occurrences([], 4) == _____
```

Avant la pause : les huit résultats attendus sont remplis. Ils ne doivent plus être modifiés pour « faire passer » un programme faux : c'est le code que l'on corrige.

6. Programmer `est_pair` et `maximum_deux` - 15 min

Ouvrir : `S3_fonctions_tests_Ahmad.py`. Compléter d'abord une fonction, exécuter ses tests, puis seulement passer à la suivante.

A. Fonction `est_pair`

1. En français : un entier est pair lorsque _____
2. Compléter le code sans affichage dans la fonction.

```
def est_pair(n):  
    """Renvoie True si n est pair, False sinon."""  
    # Remplacer pass par une instruction return.  
    pass
```

Tests à exécuter : numéros 1, 2 et 3 de la page précédente.

B. Fonction `maximum_deux`

1. Écrire le cas traité par le `if` : _____
2. Écrire le cas traité par le `else` : _____
3. Compléter le code. Les deux chemins doivent renvoyer une valeur.

```
def maximum_deux(a, b):  
    """Renvoie le plus grand des deux nombres a et b."""  
    if _____:  
        return _____  
    else:  
        return _____
```

Tests à exécuter : numéros 4, 5 et 6.

Preuve à conserver dans le fichier : six assertions placées après les fonctions. Elles doivent toutes s'exécuter sans interruption.

C. Explication orale préparée

Choisir une fonction et compléter : « Elle reçoit... ; elle renvoie... ; le cas limite testé est... »

7. Programmer `compte_occurrences` - 15 min

Spécification. Parcourir toute la liste, compter les éléments égaux à `cible`, puis renvoyer le compteur. La liste vide doit renvoyer 0 sans provoquer d'erreur.

A. Trace avant code

Pour `valeurs = [2, 5, 2, 2]` et `cible = 2`, compléter :

Tour	valeur	égal à la cible ?	compteur après le tour
1			
2			
3			
4			

B. Plan en quatre lignes

1. Initialiser _____
2. Pour chaque _____
3. Si _____
4. Renvoyer _____

C. Code à compléter

```
def compte_occurrences(valeurs, cible):  
    """Renvoie le nombre d'occurrences de cible dans valeurs."""  
    compteur = _____  
    for valeur in _____:  
        if _____:  
            compteur += _____  
    return _____
```

D. Tests et contrôle

1. Exécuter les tests 7 et 8.
2. Ajouter un neuvième test avec une cible absente d'une liste non vide.
3. Comparer le résultat du test 7 avec le comptage manuel de la table.

Neuvième assertion :

Checkpoint des trois fonctions : ☐ `est_paire` ☐ `maximum_deux` ☐ `compte_occurrences`. Le fichier s'exécute sans erreur : ☐ oui ☐ pas encore.

8. Module final et tableau d'exécution - 10 min

A. Ordre obligatoire dans le fichier

Numéroter de 1 à 4 :

- ☐ docstring générale du fichier ;
- ☐ définitions des trois fonctions ;
- ☐ assertions ;
- ☐ éventuels affichages de démonstration placés à la fin.

B. Exécution contrôlée des tests

N°	Test	Attendu	Obtenu	OK
1	<code>est_pair(8)</code>			☐
2	<code>est_pair(7)</code>			☐
3	<code>est_pair(-4)</code>			☐
4	<code>maximum_deux(3, 9)</code>			☐
5	<code>maximum_deux(-4, -9)</code>			☐
6	<code>maximum_deux(5, 5)</code>			☐
7	<code>compte_occurrences([2,5,2,2], 2)</code>			☐
8	<code>compte_occurrences([], 4)</code>			☐
9	test personnel			☐

C. Contrôle de lisibilité

Critère	Validé
Chaque fonction a un nom précis et une docstring.	☐
Aucune fonction de calcul ne dépend d'un <code>print</code> .	☐
Chaque chemin possible atteint un <code>return</code> .	☐
Les variables locales ont un rôle explicable.	☐
Les assertions restent dans le fichier après la séance.	☐
Le fichier est enregistré sous le nom demandé.	☐

Sauvegarde : enregistrer une copie datée, par exemple `S3_fonctions_tests_Ahmad_2026-08-XX.py`. Ne pas écraser la version qui fonctionne.

9. Débogage : reproduire, localiser, corriger - 12 min

Règle : ne pas réécrire toute la fonction. Pour chaque bug, prévoir le symptôme, lancer le test qui le révèle, puis modifier une seule ligne.

Bug 1 - le résultat est affiché mais pas renvoyé

```
def maximum_deux(a, b):
    if a >= b:
        print(a)
    else:
        print(b)
```

1. Test qui révèle le bug : _____
2. Valeur obtenue par l'appel : _____
3. Correction minimale : _____

Bug 2 - le compteur est réinitialisé

```
def compte_occurrences(valeurs, cible):
    for valeur in valeurs:
        compteur = 0
        if valeur == cible:
            compteur += 1
    return compteur
```

1. Plus petit exemple qui montre le bug : _____
2. Première ligne placée au mauvais endroit : _____
3. Où doit-elle être déplacée ? _____

Journal de débogage

Symptôme	Hypothèse	Modification unique	Résultat

Aides graduées - une seule à la fois : A - nommer les entrées et la sortie ; B - distinguer **print** et **return** ; C - repérer les variables locales ; D - écrire la précondition et la postcondition ; E - ajouter un cas normal, un cas limite et un cas invalide.

10. Trace écrite, exit ticket et bilan - 8 min

Trace à conserver

- Un **paramètre** est un nom dans la définition ; un **argument** est une valeur lors de l'appel.
- **print** affiche ; **return** transmet une valeur et termine l'appel courant.
- Sans **return** exécuté, une fonction renvoie **None**.
- Une variable créée dans une fonction est locale à cet appel.
- Une assertion compare un résultat obtenu à un résultat attendu ; plusieurs tests diminuent le risque de bug sans prouver l'absence de toute erreur.

Exit ticket - sans ordinateur ni aide

1. Dans `def carre(n):`, quel est le paramètre ? Dans `carre(7)`, quel est l'argument ?

2. Quelle valeur renvoie une fonction qui exécute seulement `print(5)` ?

3. Compléter avec une instruction unique :

```
def triple(n):
    -----
```

4. Écrire une assertion qui vérifie que `triple(4)` produit 12.

5. Expliquer en une phrase le rôle d'un test sur une liste vide.

Auto-évaluation

Je peux...	Pas encore	Avec aide	Seul	Expliquer
Distinguer paramètre et argument	☐	☐	☐	☐
Distinguer <code>print</code> , <code>return</code> et <code>None</code>	☐	☐	☐	☐
Écrire une fonction courte	☐	☐	☐	☐
Prévoir et écrire des assertions	☐	☐	☐	☐
Déboguer avec une modification à la fois	☐	☐	☐	☐

Le contrôle que je rendrai systématique : _____

La fonction que je peux expliquer sans lire le code : _____